Wikidown Documentation

Generated 2026-08-15

Table of Contents

Home

Wikidown is a structured markdown wiki that lives in `/docs` of any git repo — a C# CLI, an MCP server for AI agents, a Blazor WASM browser editor, and a marketing site, all built on the same page model. This wiki (the one you're reading) is itself a Wikidown wiki, dogfooding the format and tools it documents.

Where to go

- Getting Started — what a Wikidown wiki looks like on disk, the on-disk format spec, and how a wiki stays current
- CLI — the `wikidown` dotnet tool: list / read / write / move / reorder / search / check-links / backfill-breadcrumbs
- MCP Server — `wikidown-mcp`, the stdio MCP server AI agents use to read and edit a wiki
- Editor — the browser-based Blazor WASM editor that commits straight to your repo
- Agents — drop-in Claude Code and GitHub Copilot configs for maintaining a Wikidown wiki with an AI agent
- Testing — test plans and QA checklists for Wikidown's own public surfaces
- Meta — build-log notes on how Wikidown itself gets built

Getting Started

Welcome to Wikidown. This folder is itself a Wikidown-format wiki.

Format quick reference

- Page file on disk: `Getting-Started.md`; rendered title: `Getting Started`.
- Subpages go in a folder with the same base name next to the page file.
- Ordering lives in `.order` — one page base-name per line.
- Body links are relative file paths (with the `.md` extension), not absolute title paths — see Format for the full rule.

Next

- Format — full on-disk format reference
- Visual Studio Extension — add a Wikidown project in VS 2022+
- Updating — how downstream repos pick up new CLI/MCP releases, agent config changes, and VS extension updates
- CLI — `wikidown` dotnet tool
- MCP Server — `wikidown-mcp` for AI agents
- Editor — Blazor WASM browser editor
- Agents — drop-in Claude + Copilot configs
- Testing — test plans and QA checklists for Wikidown's own public surfaces
- Meta — build-log notes on how Wikidown itself gets built

Format Specification

Wikidown's on-disk format is deliberately minimal: markdown pages, folder-based hierarchy, and `.order` navigation files. The goal of this specification is to ensure that a repository's documentation is equally readable by humans browsing the file system, AI agents using the MCP server, and web-based renderers.

By enforcing these rules, Wikidown prevents the link rot and structural drift that typically plagues flat-file documentation.

1. Page Files and Titles

Every page in the wiki is a standard Markdown file (`.md`). The title of the page is derived directly from its filename by replacing hyphens with spaces.

- **File on disk:** `Release-Notes.md`
- **Rendered title:** `Release Notes`

The reverse is also true: when creating a page titled "Getting Started", the file must be named `Getting-Started.md`.

2. Subpages and Hierarchy

Wikidown supports infinite nesting of pages. To create subpages for a given parent page, you must create a folder with the exact same base name as the parent page's file, located in the same directory.

- **Parent page:** `/docs/Architecture.md`
- **Subpage folder:** `/docs/Architecture/`
- **Child page:** `/docs/Architecture/Data-Model.md`

This sibling-folder structure ensures that deleting or moving a parent page can easily include all of its children.

Every subpage folder's parent page should exist and should **link every child in its body** — see § Index pages below. `WikiRepository.Write` will happily create `/Architecture/Data-Model` even if `/Architecture` doesn't exist yet, which silently orphans the whole subtree: `wikidown list` / `wiki_search` / `wikidown check-links`' normal link scan all walk the wiki by descending from already-discovered pages, so a page whose parent was never created is invisible to all of them. `check-links` catches this specific case — see below.

3. Ordering

Alphabetical sorting is rarely the correct way to read documentation. Wikidown uses `.order` files to explicitly define the navigation hierarchy.

Every folder in the wiki (including the root `/docs` folder) should contain an `.order` file. This is a plain text file that lists the base names of the pages in that folder, one per line, from top to bottom.

Example `.order` file:

```
Getting-Started
Architecture
API-Reference
```

If a page exists in the folder but is not listed in the `.order` file, it is typically appended to the end of the list alphabetically by the renderer.

`.order` controls navigation-widget ordering only — it has no effect on GitHub's raw file view, so it's never a substitute for real body links. See § Index pages.

4. Internal Links

Internal links between wiki pages must be **relative file paths**, not absolute title paths. GitHub resolves an absolute path like `/Architecture/Data-Model` against the *repository* root, not the

wiki root, so a link written that way 404s when the page is viewed directly on github.com. A relative path resolves correctly both on GitHub and in Wikidown-aware renderers.

Write the link relative to the **linking page's own folder**, adjusted for depth, and include the `.md` extension:

- **Correct (sibling page):** `[Read the Data Model](Data-Model.md)`
- **Correct (page in a different folder):** `[Read the Data Model](../Architecture/Data-Model.md)`
- **Incorrect:** `[Read the Data Model](/Architecture/Data-Model)`

Images and other repo assets follow the same rule, e.g. `![map](../.attachments/map.png)`.

Run `wikidown check-links` (see CLI) to walk every page and verify that relative links and image references resolve to real files; by default it also flags any absolute title-path links left in page bodies, and audits that every folder has a linked index page (§ Index pages).

This rule only applies to links **inside page bodies**. Addressing a page through a tool or the CLI — e.g. `wikidown read --path /Getting-Started/Format`, `wiki_read --path /Getting-Started/Format` — still uses the absolute title path, since that's a tool argument rather than a rendered link.

Fixing `check-links` failures

`check-links` prints one line per issue:

```
page:line -> target (reason)
```

What to do depends on the `(reason)`:

- **`(absolute title-path link (404s on GitHub))`** — the link uses a `/Title/Path` form instead of a relative `.md` path. Rewrite it relative to the *linking page's own folder*, with the `.md` extension. For example, if `/Foo.md` (at the wiki root) contains `[Bar](/Bar)` and `Bar.md` is also at the wiki root, change it to `[Bar](Bar.md)`. If `/Sub/Foo.md` links to root-level `/Bar`, it becomes `[Bar](../Bar.md)`. Count the folder hops between the linking page and the target page to get the right number of `../`.
- **`(broken link)`** — a relative link/image that doesn't resolve to a real file. This is either a typo in the relative path or a stale link left over from a page that moved or was deleted. Open the linking page's folder and confirm the target filename, hop count, and `.md`/`.attachments` spelling; fix the path to match the real file. If the target page genuinely no longer exists, either remove the link or point it at wherever that content now lives.
- **A link that broke because a page moved** — `wikidown move` / `wiki_move` (see CLI and MCP Server) automatically rewrite inbound links and the moved page's own relative links when they change a page's path or folder depth. So a `check-links` failure pointing at a page that was clearly moved usually means one of two things: the move happened before that link-rewriting behavior shipped, or the link was added by hand (e.g. typed into a body) after the move rather than being created through `move`/`wiki_move`. Either way, fix it the same way as a broken relative link above — retarget it at the page's current path and depth.
- **`(no index page <Folder>.md)`** — a subpage folder exists but its sibling parent page is missing. Create it (`wikidown new --path /Folder` or `wiki_new`), then link every child from its body — see § Index pages.
- **`(not linked from parent)`** — the parent page exists but its body never links this child, so a reader browsing rendered markdown (or the raw file on GitHub, where `.order` means nothing) has no way to reach it. Add a relative link to the child in the parent's body — see § Index pages.

After editing, re-run `wikidown check-links` to confirm the line is gone.

5. Index Pages

A folder's parent page is that folder's **index** — the entry point a reader lands on before descending into its children. Two invariants keep that entry point real rather than aspirational, both audited by `wikidown check-links` (pass `--no-index-check` to skip this pass):

- **The index page must exist.** `/Architecture/Data-Model` can be created without `/Architecture` ever existing — nothing in `WikiRepository.Write` requires the parent first. When that happens the whole subtree becomes invisible to every wiki-model-based tool (`wikidown list`, `wiki_search`, and `check-links`' own link scan), since they all discover pages by descending from an already-discovered parent. `check-links` finds these orphans by walking the raw filesystem instead, specifically because it can't rely on the page model to see them.
- **The index page must link every child.** `.order` decides navigation order in Wikidown-aware UIs, but it's invisible on GitHub's raw file view — the only way a reader following links on github.com can reach a child page is a real link in the parent's body. `check-links` verifies every `*.md` file directly inside a folder is the resolved target of at least one link (relative or absolute) in that folder's parent page.

A page's own breadcrumb links upward to its ancestors, but that's a different direction from this check: the breadcrumb doesn't help a reader on the *parent* page discover its children, and it doesn't verify the ancestor pages it links to actually exist — a folder missing its index page still gets a breadcrumb pointing at a `.md` file that isn't there, which is exactly the case index-page auditing is meant to catch.

6. Breadcrumb Navigation

Every page gets a one-line breadcrumb trail auto-injected as its **first line**, always leading back to `/Home` when the wiki has one:

```
[Home](../Home.md) / [Encounters](../Encounters.md) / The Sky Hunters <!--
wikidown:breadcrumb -->
```

This exists because GitHub's own file-path breadcrumb (shown above the raw file view) reflects the *repository* path — `docs / Encounters / The-Sky-Hunters.md` — not the wiki's page hierarchy or titles. Wikidown's breadcrumb is built purely from the page's own title chain instead: `Home` (if it exists) leads, then each further ancestor is a link (relative, per the convention above), and the current page's title is the final, unlinked segment — the same shape GitHub uses, just scoped to the wiki rather than the whole repo.

Key behavior:

- **Automatic.** `wikidown write` / `wiki_write` (and new / `wiki_new`, which write through the same path) inject or refresh the breadcrumb on every save — there's nothing to opt into or maintain by hand. Write whatever body content you want; the first line is managed for you.
- **Idempotent.** The line carries an HTML comment marker (`<!-- wikidown:breadcrumb -->`) that's invisible when rendered. On every write, whatever was on line one gets discarded if it carries that marker, then a fresh one is computed from the page's *current* path — so re-saving a page never accumulates duplicate breadcrumbs, and a round-tripped `read` → edit → `write` doesn't need to preserve the line itself.
- **Always leads back to `/Home`, when the wiki has one.** `wikidown init` seeds a `/Home` page as the wiki's entry point; every other page's breadcrumb — including top-level pages, which otherwise have no ancestors — leads with a link to it, so there's always a way back. A wiki with no `/Home` page (this repo's own `/docs` predates that convention) never gets a fabricated link to a page that doesn't exist: top-level pages simply have no breadcrumb, and nested pages' breadcrumbs start from their nearest real ancestor, same as before `/Home` was introduced. `/Home` itself never links to itself.
- **Moves regenerate it, not just patch it.** `wikidown move` / `wiki_move` fully regenerates the breadcrumb for the moved page and every moved descendant, rather than trying to edit the existing links in place — a move can change *which* ancestors a page has, not just how many `../` hops reach them, so a patch-in-place approach can leave a structurally wrong (if still technically valid) breadcrumb behind.
- **Checked like any other link.** Breadcrumb links are ordinary relative markdown links, so `check-links` validates them the same way it validates everything else in the body — but only that they *resolve*. Whether the ancestor page they point at *should* exist and is properly indexed is § Index Pages' job, not this one.
- **Wikis that predate `/Home`, or predate breadcrumbs entirely,** won't have this line until each page is re-saved. Run `wikidown backfill-breadcrumbs` once to catch every page up in one pass — see CLI.

7. Markdown Dialect

Wikidown relies on standard CommonMark. There are no proprietary macros or shortcodes required to render the core text. An MVP renderer only needs a standard markdown parser plus the filename↔title mapping logic described above.

Visual Studio Extension

The **Wikidown** Visual Studio extension surfaces your wiki's `docs/` folder directly in Solution Explorer — without it participating in the build.

Install

1. Open **Extensions → Manage Extensions** in Visual Studio 2022 or later.
2. Search for **Wikidown** in the Marketplace tab and click **Install**.
3. Restart Visual Studio when prompted.

Alternatively, download `Wikidown.Vs.vsix` directly from the Visual Studio Marketplace and double-click to install.

Add a Wikidown project to your solution

1. Right-click the solution node in **Solution Explorer** and choose **Add → New Project**.
2. Search for **Wikidown** (or filter by project type **Wikidown**).
3. Select **Wikidown Wiki** and click **Next**.
4. Give the project a name (default: `wiki`) and pick any folder you like — the location field defaults sensibly — then click **Create**.

When the project is created, the extension looks for an existing `docs/` folder near the project file (checking the project folder and up to two parent folders). If it finds one, the new project points at it; if not, it creates a `docs/` folder next to the project file seeded with a starter `Home.md` page. The wiki folder appears immediately under the new project node in Solution Explorer.

Configure the wiki root

By default the project looks for a `docs/` folder next to the `.wikidownproj` file. Edit the file to point to a different location:

```
<WikidownProject>
<WikiRoot>my-wiki</WikiRoot> <!-- relative to the .wikidownproj file -->
</WikidownProject>
```

Features

• Presents the wiki the way Azure DevOps does: nodes show page **titles** (dashes render as spaces, no `.md` extension), a page with a same-named subpage folder is a single expandable node, `.order` files are hidden, and pages sort by `.order` (listed entries first, then alphabetical).

• Double-clicking a page opens it in VS's built-in markdown editor.

• Right-click the project, a folder, or a page for the standard context menus, including the **Git** submenu, plus wiki-native commands:

  • **Add Page…** prompts for a name, creates the page, and records it in `.order`.

  • **Add Folder…** creates a folder plus a same-named page beside it, following the CLI's subpage convention.

  • **Add Existing Pages…** copies `.md` files in and records them.

  • **Move Page Up** / **Move Page Down** reorder the page within its folder's `.order`.

  • **Edit Page Order** opens the folder's `.order` in the editor (creating it from the current display order if missing).

  • **Delete Page** (a page and, for pages with children, its subpages) and **Delete Folder** confirm and then delete — your git history is the undo.

• Everything stays consistent with the `wikidown` CLI and the `wiki_*` MCP tools, which remain fully usable side by side — the tree refreshes live when they change files or reorder `.order`.

• The project node is **display-only** — it never appears in Build, Rebuild, or Clean, and adds no compile items to the solution.

Add an existing project to a `.sln` manually

If you prefer to hand-edit the solution file, use the Wikidown project type GUID:

```
Project("{6a9c3f4b-d5e8-4f0a-b1c2-345678901bcd}") = "wiki", "wiki.wikidownproj",
"{<new-guid>}"
EndProject
```

Related

- CLI — `wikidown` dotnet tool for editing from the terminal or a CI pipeline.
- MCP Server — stdio MCP server for AI agents (Claude, Copilot).
- Editor — browser-based Blazor editor that commits directly to your repo.

Updating

How a downstream repo picks up new Wikidown releases — CLI/MCP tool versions, agent config files, and the VS extension each update differently.

CLI and MCP server (NuGet global tools)

`Wikidown.Cli` and `Wikidown.Mcp` are published to NuGet as global .NET tools. Update them with:

```
dotnet tool update -g Wikidown.Cli
dotnet tool update -g Wikidown.Mcp
```

A push to `main` doesn't always mean a new version is available. The release.yml workflow runs on every push to `main` that touches `Directory.Build.props`, `src/Wikidown.Cli/**`, `src/Wikidown.Mcp/**`, `src/Wikidown.Core/**`, `assets/**`, or the workflow file itself, and it packs + pushes to NuGet with `--skip-duplicate`. The package version comes from <VersionPrefix> in `Directory.Build.props` (currently `0.2.1`) — and that value is **not** auto-incremented per push. `--skip-duplicate` means a push that doesn't bump `VersionPrefix` re-packs and tries to push the *same* version, which NuGet rejects as a duplicate and the step just no-ops. So a plain code-fix push only becomes something `dotnet tool update` can see once a maintainer bumps `VersionPrefix` in the same push (or a later one).

If `dotnet tool update` reports you're already on the latest version but you expected a fix, check whether `VersionPrefix` has actually moved since your last update — the fix may be merged but not yet released.

Agent configs (`.claude/`, `.github/`, `.vscode/mcp.json`, `CLAUDE.md`)

These files are copied into a downstream repo once, by `wikidown init` (see CLI and Agents) — they don't auto-update. To pick up changes to the shipped configs (wording tweaks, new instructions, new tool wiring), re-run `init` with `--force`:

```
wikidown init --agents all --force
```

`--force` is required because `init` otherwise skips any file that already exists in the target repo. Review the diff before committing — `--force` overwrites local edits to those files too.

VS extension (VSIX)

The Visual Studio extension auto-updates through the **Visual Studio Marketplace**'s normal extension update mechanism once a new version is published. Publishing is triggered by pushing a `vsix-v*` tag (see vsix.yml), which builds the VSIX, runs `VsixPublisher.exe` against the Marketplace, and attaches the `.vsix` to the GitHub Release. No action is needed downstream beyond letting Visual Studio check for extension updates as usual — see Visual Studio Extension.

Summary

Component	Trigger to publish	How consumers update
`Wikidown.Cli` / `Wikidown.Mcp` (NuGet)	Push to `main` touching Core/CLI/MCP/ `Directory.Build.props`, **and** a `VersionPrefix` bump	`dotnet tool update -g Wikidown.Cli` / `Wikidown.Mcp`
Agent configs	N/A — copied at `init` time	`wikidown init --agents all --force`
VS extension (VSIX)	Push of a `vsix-v*` tag	Automatic via VS Marketplace update check

CLI

The `wikidown` dotnet tool reads, writes, moves, reorders, searches, and link-checks pages in a Wikidown wiki. It keeps `.order` files, breadcrumb navigation, and (on `move`) links in sync automatically.

Install

```
dotnet tool install -g Wikidown.Cli
```

To update an existing install, see Updating — a plain push to `main` doesn't always mean a new NuGet version is available.

Wiki root

The CLI defaults to `./docs`. Override with `--root <path>`:

```
wikidown --root ./my-wiki list
```

Commands

- `init [--root docs] [--agents claude|copilot|all|none] [--force]` — seed an empty wiki with a `/Home` page and install the AI agent configs (Claude Code
  - GitHub Copilot) into the folder containing the wiki root. See Agents. Re-run with `--force` to pick up updated agent configs in an existing repo — see Updating.
- `list [--path /P]` — list children of a page (or root). `wikidown list`
- `read --path /P` — print page markdown to stdout. `wikidown read --path /Getting-Started`
- `write --path /P [--file F | --stdin]` — overwrite a page. Auto-injects or refreshes the page's breadcrumb line — see Format § Breadcrumb Navigation.
- `new --path /P [--title T] [--file F | --stdin]` — create a new page.
- `move --from /A --to /B [--dry-run]` — rename or move a page (subpages travel with it). Rewrites inbound links from every other page that pointed at the old path, rewrites the moved page's own relative links and images if the move changed its folder depth (e.g. moving `/Encounters/Foo` to `/Adventures/Bar/Foo` turns `../.attachments/x.png` into `../../.attachments/x.png`), and regenerates the breadcrumb for the moved page and every moved descendant. Reports a count and a per-link list of what changed. `--dry-run` previews the rewrite without touching any files.
- `delete --path /P [--recursive]` — delete a page (and optionally its subpages).
- `reorder --folder /P --names a,b,c` — rewrite `.order` for a folder.
- `search --query <text>` — full-text search across page bodies.
- `check-links [--no-absolute-check] [--no-index-check]` — walk every page and validate that relative markdown links (`[x](../Foo/Bar.md)`) and image references (`![x](../.attachments/pic.png)`) resolve to real files relative to the linking page's folder. By default also:
  - flags absolute title-path links in page bodies (`[x](/Foo/Bar)`), since GitHub resolves those against the repo root and they 404 when the wiki is browsed on github.com (`--no-absolute-check` to skip);
  - audits that every subpage folder has an index page and that the index page links every child in its body, since `WikiRepository.Write` can create a grandchild page without its parent ever existing, silently orphaning the subtree from `wikidown list` / `wiki_search` / the rest of `check-links` itself (`--no-index-check` to skip). See Format § Index Pages.
- `backfill-breadcrumbs [--dry-run]` — one-time catch-up for a wiki that predates breadcrumb navigation (chunk 11): re-saves every page that has an ancestor but is missing its breadcrumb line, so it picks one up. Only needed once per existing wiki — `write`, `new`, and `move` all maintain breadcrumbs automatically going forward, so a wiki that's always been edited through this CLI (or the MCP tools) never needs it. `--dry-run` lists which pages would change without writing anything. Prints a count.

- `export-pdf --output <path> [--from /Link/Path] [--title T] [--no-cover] [--no-toc] [--allow-html-skip]` — combine the whole wiki (or a subtree, with `--from`) into a single linked PDF:
  - an optional cover page (title defaults to the repo's folder name, override with `--title`, skip with `--no-cover`);
  - an optional in-document table of contents page with page numbers and clickable entries, indented to match the nav hierarchy (skip with `--no-toc`);
  - one section per wiki page, with each page's own leading `# Title` heading placed at its nav depth, so the PDF's bookmark/outline panel mirrors the wiki's folder structure instead of listing every page flat;
  - internal wiki links — relative `.md` links, legacy absolute title-paths, and same-page `#fragment` links to headings — become real in-PDF jumps, not dead hrefs;
  - full markdown fidelity: headings, bold/italic, inline code, nested bullet/numbered lists, fenced code blocks, pipe tables, and images (embedded when the relative path resolves).

See Format for the on-disk format the CLI maintains, including the relative-link convention that `check-links` enforces and the breadcrumb navigation it injects.

MCP Server

`wikidown-mcp` is a stdio MCP server that exposes the Wikidown CLI surface to Claude Code, Claude Desktop, VS Code (Copilot), or any other MCP host.

Install

```
dotnet tool install -g Wikidown.Mcp
```

To update an existing install, see Updating — a plain push to `main` doesn't always mean a new NuGet version is available.

Wiki root

Selected in this order:

1. `--root <path>` flag
2. `WIKIDOWN_ROOT` environment variable
3. Default `./docs`

Tools

- `wiki_list` — list children of a page or the root
- `wiki_read` — read a page
- `wiki_write` — overwrite a page. Auto-injects or refreshes the page's breadcrumb navigation line — see Format § Breadcrumb Navigation.
- `wiki_new` — create a new page
- `wiki_move` — rename or move a page (with subpages). Rewrites inbound links from every other page that pointed at the old path, rewrites the moved page's own relative links and images if the move changed its folder depth, regenerates the breadcrumb for the moved page and every moved descendant, and reports a count and a per-link list of what changed.
- `wiki_delete` — delete a page (optionally recursive)
- `wiki_reorder` — rewrite a folder's `.order`
- `wiki_search` — search page bodies
- `wiki_walk` — depth-first walk of every page

There's no `wiki_check_links` tool yet — run `wikidown check-links` from the CLI (see CLI) to validate that relative links/images resolve and that page bodies don't contain absolute title-path links, which 404 when a page is viewed directly on github.com.

Wiring it in

Sample configs for Claude Code (`.mcp.json`) and Claude Desktop live in samples/mcp/ in the repo. AI agents should prefer these tools over raw file edits so `.order` files and internal links stay consistent.

Editor

Wikidown ships a Blazor WebAssembly PWA that lets you browse and edit any Wikidown wiki straight from the browser. It's hosted on Azure Static Web Apps at `https://wikidown.app/` (default hostname `https://victorious-wave-03164381e.7.azurestaticapps.net/` still serves the editor, but OAuth sign-in only completes on `wikidown.app`), with a small Functions app at `/api/*` that only exists to complete OAuth token exchange — page commits still go directly to your provider's REST API from the browser.

Providers

- **GitHub** — Sign in with GitHub (OAuth) or paste a fine-grained Personal Access Token. Commits via the Contents API.
- **Azure DevOps** — Personal Access Token. Commits via the Pushes API. (OAuth is planned but not yet wired.)

Access tokens are kept in browser `localStorage`. The Functions app never sees your repo contents and never stores your token — it only swaps an OAuth code for an access token and redirects back.

Sign-in flow (GitHub)

1. Click **Sign in with GitHub** on `/connect`.
2. Redirect to `github.com/login/oauth/authorize` for the `Wikidown` OAuth App.
3. Return to `/oauth/github/return.html` — a tiny static page served by ASWA. Its JS reads `code` + `state` from the query string and `fetch`-POSTs them as a JSON body to `/api/auth/github/exchange`.
4. The `GitHubExchange` Function swaps `code` + `client_secret` for an access token against `https://github.com/login/oauth/access_token` and returns `{"token":"…"}` (or `{"error":"…"}`).
5. The relay page redirects to `/connect#gh_token=...&gh_state=...`. The SPA validates the CSRF `state`, stores the token in `localStorage`, scrubs the hash, and navigates to `/browse`.

The two-piece relay exists because Azure Functions on ASWA managed backends treats `?code=` as a reserved function-key candidate and rejects requests that carry it with a pre-dispatch HTTP 500 — even on `AuthorizationLevel.Anonymous` routes. Since GitHub's OAuth redirect always appends `?code=…`, the callback URL has to land somewhere other than `/api/*`. See Vibing Phase Recap for the war story.

A "Use a PAT instead" toggle on the same page is kept as a fallback.

GitHub OAuth Apps support a single `Authorization callback URL`, so the `Wikidown` app is registered against `https://wikidown.app/oauth/github/return.html`. Sign-in on the ASWA default hostname will fail with a `redirect_uri` mismatch — use `wikidown.app` for OAuth, or paste a PAT as a fallback.

Browse and edit

- Read mode renders pages with MudMarkdown (CommonMark + KaTeX-ready).
- Edit mode is side-by-side: textarea on the left, live preview on the right.
- Save commits the change with a configurable message.

Conflict handling

The editor records the file SHA (or the branch-head commit OID, depending on the provider) before you edit. If the remote has moved when you save, the commit fails with a "Reload remote" banner so you don't clobber someone else's change.

Drafts

In-progress edits are kept per (`provider, owner, project, repo, branch, page`) in `localStorage`. They survive page reloads and crashes, and clear on a successful commit.

Agents

Wikidown ships drop-in configs so AI coding assistants can maintain your wiki through the MCP Server without you wiring anything up by hand.

Shared skill

Claude and Copilot share one skill file in the Agent Skills standard format: agents/skills/wikidown/SKILL.md. It carries the format rules, tool cheat sheet, CLI fallback, and workflow.

- Claude Code loads it from `.claude/skills/wikidown/SKILL.md`.
- GitHub Copilot loads it from `.github/skills/wikidown/SKILL.md`.

Claude Code

In agents/claude/, layered on top of the shared skill:

- `wikidown.subagent.md` → `.claude/agents/wikidown-editor.md` — a subagent definition that owns `/docs/*.md` reads and writes.
- `CLAUDE.md` snippet — append to your project's root `CLAUDE.md` so every Claude session knows to delegate wiki edits.
- `samples/mcp/claude-code.mcp.json` → `.mcp.json` — wires `wikidown-mcp` into Claude Code.

Copilot

In agents/copilot/, layered on top of the shared skill:

- `copilot-instructions.md` → `.github/copilot-instructions.md` — repo-wide guidance.
- `wikidown.agent.md` → `.github/agents/wikidown.agent.md` — a custom agent, also used by the Copilot coding agent on github.com.
- `wikidown.chatmode.md` → `.github/chatmodes/wikidown.chatmode.md` — a custom chat mode focused on wiki editing.
- `mcp.json` → `.vscode/mcp.json` — wires `wikidown-mcp` into VS Code.

Install

The recommended install is the CLI:

```
dotnet tool install -g Wikidown.Cli
wikidown init --agents all
```

`--agents` accepts `claude`, `copilot`, `all`, or `none`. Existing files are skipped unless you pass `--force`; an existing `CLAUDE.md` gets the wiki section appended if it doesn't mention the wiki already.

Manual copying still works: the agents/README.md maps each source file to its destination. Once installed, the agents call the `wiki_*` MCP tools so `.order` files and link targets stay consistent.

Keeping configs current

These files are copied once, not auto-updated. See Updating for how to re-run `wikidown init --agents all --force` to pick up config changes shipped in newer Wikidown releases.

Testing

Test plans and QA checklists for Wikidown's public surfaces.

Pages

- Browser Test Plan — end-to-end manual or automated browser checks across the marketing site, editor PWA, and custom domain. Designed to be handed to an AI agent with a browser-automation tool (e.g. Playwright MCP).

Browser Test Plan

A handoff checklist for an AI agent (or human) driving a real browser — e.g. via Playwright MCP — to exhaustively smoke-test Wikidown's public surfaces.

How to use this page

Wikidown now has two distinct public surfaces:

1. **Marketing site** — `https://wikidown.org/` (static HTML + CSS, served by GitHub Pages; marketing only, no editor, no API).
2. **Editor PWA + API** — `https://wikidown.app/` (Blazor WASM + MudBlazor at `/`, .NET isolated Functions at `/api/*`, served by Azure Static Web Apps). The ASWA default hostname `https://victorious-wave-03164381e.7.azurestaticapps.net/` is still bound and should behave identically.

For every check, report **PASS**, **FAIL**, or **SKIPPED** with a one-line reason. On any failure, attach:

- URL at time of failure
- Viewport size
- Browser + version
- Console errors (if any)
- Network errors (if any)
- Screenshot

Checks are ordered so you can stop early on a catastrophic failure (e.g. site doesn't load).

Marketing site — `https://wikidown.org/`

Load and assets

- `GET /` returns HTTP 200 and renders without console errors.
- `<title>` is set and non-empty; favicon visible in the tab.
- Logo image loads and has **transparent corners** — no white square behind the rounded glyph on the page background. Sample pixels at the four corners of the logo element to confirm alpha = 0.
- All `<img>` / `<link rel="stylesheet">` resources return 200 (Network panel has no reds).
- No mixed-content warnings; every asset is HTTPS.
- `og:title`, `og:description`, and `og:image` meta tags are present.

Navigation

- **"Open the editor"** button → `https://wikidown.app/`.
- **"View on GitHub"** has `target="_blank"` and resolves to HTTP 200.
- Every nav / hero / footer link resolves to HTTP 200 — none are `#` or `javascript:void(0)`.
- `GET /app/` on the marketing origin returns 404 — the editor is **not** served from the marketing site anymore.

Layout and readability

- The feature-card grids are fully visible — no horizontal overflow, no text clipped, no card overlaps another.
- Each card's kicker, heading, and body text are legible.
- The "Built for humans and agents" row has three equal-width cards on desktop with aligned code blocks at the bottom.

Mobile breakpoint

- At viewport width ~480 px, grids stack into a single column without horizontal scroll.

- Hero stacks (logo above or below the copy; not overlapping).
- Hero text stays readable (no character clipped, no font <14 px).
- `document.documentElement.scrollWidth == document.documentElement.clientWidth`.

Editor PWA — `https://wikidown.app/`

Shell and first paint

- `GET /` returns HTTP 200 and the Blazor WASM app reaches interactive state (loading spinner disappears within ~10 s on a fast connection).
- App bar shows the **W↓ logo** immediately to the left of the word **Wikidown**.
- The **Drafts** button is visible in the app bar. Clicking it on a clean profile shows the empty state (no drafts).
- Home page content renders — no perma-spinner, no red error banner.
- Browser console shows no unhandled exceptions during load.

Static assets

- `GET /favicon.ico` → 200.
- `GET /apple-touch-icon.png` → 200.
- `GET /manifest.webmanifest` → 200. `Content-Type` is `application/manifest+json` (or `application/json`). Body is valid JSON and declares `name`, `short_name`, `start_url`, `display`, and at least one `icons` entry.

Service worker

- DevTools → Application → Service Workers lists a registered SW for scope `/`, status **activated and is running**.
- DevTools → Application → Cache Storage shows at least one `offline-cache-*` entry populated after a reload.
- A hard reload (`Ctrl+Shift+R`) still produces a working app; a subsequent normal reload is served partly from cache (Network panel shows some requests flagged `(ServiceWorker)` or `(disk cache)`).

API smoke tests — `/api/*`

The `/api/*` routes are served by the linked Azure Functions app. These are quick unauthenticated probes — they don't touch real OAuth.

- `GET /api/config/github` → HTTP 200, `Content-Type: application/json`, body shape `{"clientId":"..."}`. If `clientId` is empty, the SWA's `GITHUB_CLIENT_ID` application setting isn't configured yet — mark **FAIL** and stop the OAuth section below.
- `GET /api/config/ado` → HTTP 200, body shape `{"clientId":"..."}` (empty string acceptable — ADO OAuth is not wired yet).
- `POST /api/auth/github/exchange` with body `{"code":"not-a-real-code","state":"xyz"}` and `Content-Type: application/json` → HTTP 502 with body `{"error":"token_exchange_failed"}` (GitHub returns non-200 for the bad code). Confirms the token-exchange round-trip is wired.
- `POST /api/auth/github/exchange` with body `{}` → HTTP 400 with body `{"error":"missing_code"}`.
- `GET /oauth/github/return.html` → HTTP 200, serves the static relay page (briefly shows "Completing GitHub sign-in…" before redirecting to `/connect#gh_error=missing_code` because no code is in the query string).
- **Do not** add `?code=…` to any `/api/*` URL when probing — Azure Functions on ASWA's managed backend rejects `?code=` queries pre-dispatch with HTTP 500, regardless of `AuthorizationLevel`. The relay-via-static-page flow exists specifically to avoid this. See `/Meta/Vibing-Phase-Recap`.

GitHub OAuth sign-in flow *(skip by default)*

Only run if a throwaway GitHub account is available to approve the `Wikidown` OAuth App. Without one, mark every check in this section **SKIPPED**.

With throwaway credentials:

- Navigate to `/connect`. The GitHub tab shows a **Sign in with GitHub** button as the primary action, *not* a PAT field by default. A "Use a PAT instead" toggle is visible.
- Fill Owner + Repo for a repo the throwaway account can access.
- Click **Sign in with GitHub**. Browser leaves the app and lands on `github.com/login/oauth/authorize?...` with the `Wikidown` app name visible on the consent screen.
- Approve. Browser returns through `/oauth/github/return.html?code=...&state=...`, briefly shows "Completing GitHub sign-in..." while the relay page POSTs the code to `/api/auth/github/exchange`, then lands on `/connect` with the hash immediately cleaned up by `history.replaceState` (inspect the address bar — no #gh_token= left behind after first paint).
- The app auto-navigates to `/browse` and the snackbar reads "Connected to [html]/[html]".
- DevTools → Application → Local Storage has key `wikidown.connection.v1` with a JSON value that includes `token` (the access token) and `provider: "GitHub"`.
- DevTools → Application → Session Storage is empty — `wikidown.gh_state` and `wikidown.gh_pending` have been cleared.
- Returning to `/connect` shows the "Connected: ..." banner with **Browse** and **Disconnect** buttons.
- Clicking **Disconnect** removes `wikidown.connection.v1` from Local Storage and the connected banner disappears.

State-mismatch negative test:

- Start a sign-in, then manually set `sessionStorage['wikidown.gh_state']` to a different value before returning from GitHub. The `/connect` page should show a "state mismatch" error and **not** store a connection.

ASWA default hostname — `https://victorious-wave-03164381e.7.azurestaticapps.net/`

The vanity domain is `wikidown.app`, but the ASWA default hostname stays bound as a safety net. Every check above should behave identically here **except GitHub OAuth sign-in**, which only completes on `wikidown.app`.

- `GET /` returns 200 and reaches interactive state.
- TLS certificate is valid (Azure-managed, no browser interstitial).
- `/api/config/github` and `/api/ping` respond the same as on `wikidown.app`.
- **GitHub OAuth sign-in is expected to FAIL here.** GitHub OAuth Apps support a single `Authorization callback URL`, which is registered against `https://wikidown.app/oauth/github/return.html`. Starting sign-in from the ASWA default hostname should produce a `redirect_uri` mismatch error from `github.com/login/oauth/authorize`. The PAT fallback still works here.

PWA install

Run in a Chromium-based browser (Chrome or Edge) where install prompts are supported.

- At the editor origin, the install affordance appears — either a native install prompt fires, or the install icon shows in the address bar.
- Triggering install opens a standalone app window.
- The installed PWA's taskbar / dock / launchpad icon is the Wikidown W↓ icon, not a generic globe.
- Launching the installed PWA opens without browser chrome and lands on the editor Home page.

- Uninstalling cleanly removes the icon and leaves no zombie window.

Responsive and theme

Viewport widths

Test both live surfaces (marketing + editor) at:

- **360 px** (small phone) — no horizontal scroll, tap targets ≥ 40 px, all text readable.
- **768 px** (tablet) — layout adjusts, no awkward large gaps, nav still usable.
- **1280 px** (laptop) — full desktop layout, no stretched hero image, content max-width looks intentional.

Landscape on mobile

- At 812×375 (iPhone landscape), the editor app bar stays visible and consumes ≤ ~15% of vertical space.
- Marketing hero does not push the CTA below the fold unreachably.

Theme

- Toggle OS-level dark mode. The marketing site and editor either follow it or stay in their chosen theme — but never produce unreadable dark-on-dark or light-on-light text.
- No flash of wrong theme (FOUC) on reload.

Regression checklist

A 10-item fast pass for any future deploy:

1. `https://wikidown.org/` returns 200 and renders.
2. Logo has transparent corners on the marketing site.
3. "Open the editor" button reaches `https://wikidown.app/`.
4. The editor origin returns 200 and reaches interactive state with no console errors.
5. App bar shows `W↓ Wikidown` with Drafts menu visible.
6. `manifest.webmanifest` returns 200 and is valid JSON.
7. Service worker is registered and activated.
8. `GET /api/config/github` returns 200 with a non-empty `clientId`.
9. `GET /oauth/github/return.html` returns 200 (static relay page) and `POST /api/auth/github/exchange` with `{}` returns 400 `{"error":"missing_code"}`.
10. 480 px mobile layout stacks without horizontal scroll.

Known gaps

- **GitHub OAuth flow requires throwaway credentials** and an approved consent on the `Wikidown` OAuth App — SKIPPED without them.
- **ADO OAuth is not wired yet.** `/api/config/ado` returns an empty `clientId` on purpose; ADO tab still uses PAT.
- **No device-flow test.** GitHub Device Flow isn't implemented yet; it's only useful for CLI / MCP, not for this browser test.
- **No accessibility audit here.** A11y (axe, Lighthouse a11y) should live in a dedicated page.
- **No performance budgets.** Lighthouse scores are informational; thresholds aren't defined yet.

Reporting format

When you finish, produce a short summary like:

```
Marketing site: 12/12 PASS
Editor PWA shell: 9/9 PASS
API smoke tests: 4/4 PASS
GitHub OAuth: SKIPPED (no throwaway creds)
Custom domain: SKIPPED (not live)
PWA install: 4/5 PASS (1 failure, see screenshot)
Responsive/theme: 7/7 PASS
Regression (10): 10/10 PASS
```

Attach every failure screenshot and a combined HAR of both surfaces.

Meta

Behind-the-scenes notes about how Wikidown itself gets built — build-log posts, phase recaps, architectural decisions that didn't fit in the main docs.

Pages

- Vibing Phase Recap — build log from getting the WASM editor, Functions API, marketing site, and CLI/MCP tool pair standing up.

Wikidown — "Vibing Phase" Recap

A build log / field notes of getting the basics standing up, from nothing to a deployed WASM editor with a working Functions API, a marketing site on its own domain, and a CLI + MCP tool pair.

The shape of the thing

Wikidown is a structured markdown wiki that lives in `/docs` of any git repo — no service to run, tokens never leave the browser. One codebase ships four surfaces:

- `Wikidown.Core` — the page model: `.order` files, title-form links (`/Parent/Child`), on-disk read/write.
- `Wikidown.Cli` — a `dotnet tool` called `wikidown` for list / read / write / move / search.
- `Wikidown.Mcp` — an MCP stdio server exposing the same surface to Claude Code, Claude Desktop, and VS Code Copilot.
- `Wikidown.Web` — a Blazor WASM + MudBlazor PWA editor.
- `Wikidown.Site` — a static marketing page.
- `Wikidown.Api` — a .NET isolated Azure Functions app for the OAuth token exchange.

The wiki eats its own dogfood: this repo's own `/docs` is a Wikidown wiki, edited through the `wikidown-editor` subagent and the `wikidown` skill.

Deployment topology we landed on

After one wrong turn (more below), we split the surfaces cleanly:

- **`wikidown.org`** — GitHub Pages, marketing only. Plain HTML/CSS, zero backend, Porkbun DNS with four A records to GitHub's 185.199.108-111.153.
- **`wikidown.app`** — Azure Static Web Apps. Blazor WASM editor at `/`, managed Functions at `/api/*`. The ASWA default hostname `victorious-wave-03164381e.7.azurestaticapps.net` stays bound as a safety net.

GitHub Pages is the wrong host for OAuth (no runtime), so anything that needs a server — token exchange, config endpoints — lives on ASWA. Marketing-only stays on Pages because it's free, cached globally, and wiring a .NET WASM app into it gets you nothing.

GitHub OAuth, the real way

The first cut of `/connect` was PAT-only with a frozen "GitHub OAuth Device Flow needs a small proxy…" caption. That caption was still true — a proxy was needed — so we built one.

- **`/api/config/github`** — returns the public `clientId` (empty string means the SWA app setting isn't wired).
- **`/oauth/github/return.html` + `/api/auth/github/exchange`** — a static HTML relay page is the registered OAuth callback. Its JS reads `code` + `state` from the query string and POSTs them as a JSON body to the `GitHubExchange` Function, which swaps `code` for a token against `https://github.com/login/oauth/access_token` using the server-side `GITHUB_CLIENT_SECRET` and returns `{"token":"…"}`. The page then redirects to `/connect#gh_token=…&gh_state=…`. The reason for the relay (rather than a single Function callback) is documented in §6 below.
- **`/connect` in the WASM app** — generates a CSRF `state`, stashes pending form + state in sessionStorage, redirects to `github.com/login/oauth/authorize`, receives the fragment on return, validates state, stores the access token in localStorage, and scrubs the fragment via `history.replaceState` so no token is ever left in the address bar. A "Use a PAT instead" toggle keeps the old path available.

Device Flow was consciously deferred. It's the right answer for the CLI / MCP surfaces, not for a browser.

Things that went sideways, and what they taught us

1. .NET 10 vs the world

The repo is on .NET 10 (pinned `10.0.100` in `global.json`) because it's fun and we can. Azure's Oryx builder — the image that powers `Azure/static-web-apps-deploy` — doesn't have a .NET 10 SDK yet. First ASWA run exploded with "*Requested SDK version 10.0.100 … Installed SDKs: [none match]*".

Pattern that worked:

- **Pre-build the Blazor WASM editor on the GitHub runner** where `actions/setup-dotnet@v4` can install any SDK, then hand the published `wwwroot` to the deploy action with `skip_app_build: true`.
- **Let Oryx build the Functions API**, but drop a scoped `src/Wikidown.Api/global.json` pinning `9.0.x` with `rollForward: "latestMinor"` so Oryx picks its bundled 9.0 SDK instead of tripping on the root 10.0 pin. Letting Oryx do the API build also means it writes the `functions.metadata` file the deploy step needs to register functions.

Lesson: when you're ahead of the hosted build image, do the build on the runner and make the deploy action a pure uploader.

2. Functions language version

Oryx's first API-side error was unusually helpful: "*dotnetisolated versions are valid: 8.0, 9.0*". We'd targeted net10. Retargeted `Wikidown.Api` alone to net9, everything else stayed on net10. Production net9 artifacts on a net9 isolated worker, no drift.

We also briefly had the API global.json on `latestMajor` — Oryx dutifully downloaded SDK 10.0.200 to build a net9 project. That's the drift we didn't want; tightening to `latestMinor` kept SDK and TFM aligned.

3. The service worker ate our API

Most satisfying bug of the phase. After deploy, every API function registered cleanly — the portal showed all five. But hitting `/api/ping` in a browser tab loaded the Blazor shell and rendered Blazor's own "Not Found" page. The `staticwebapp.config.json` correctly excluded `/api/*` from navigation fallback, so SWA was blameless.

The Blazor PWA service worker — the one template ships with — intercepts `mode === 'navigate'` fetches, finds the URL isn't in the asset manifest, and cheerfully serves cached `index.html`. The request never left the browser. Our `/api/*` route excludes were configured on the wrong layer.

Fix was four lines: in `service-worker.published.js`, detect same-origin `/api/*` and `return` from the fetch handler *before* calling `event.respondWith`, so the browser makes a plain network request SWA can route.

Lesson: in a Blazor PWA + co-located API setup, the service worker is the *first* router the request hits. Its exclude list is more important than any `staticwebapp.config.json` rule, because SWA never even sees the request if the SW shortcuts it.

4. The Porkbun ALIAS detour

`wikidown.org` briefly broke because Porkbun was serving an `ALIAS → markdav-is.github.io` record. GitHub Pages apex domains want four plain A records (185.199.108.153 / .109.153 / .110.153 / .111.153). Swapping them in made `NotServedByPagesError` go away and Let's Encrypt issued the cert within ~15 minutes.

5. The "(managed)" red herring

Azure Portal's SWA → APIs blade shows a blue banner: "*Bring your own API backends are not supported in the Free hosting plan. Click to upgrade.*" Right below it, Production → Function App → `(managed)`. Those two things look contradictory but aren't — the banner only applies to BYO backends (external Function App, Container App, API Management). The managed Functions backend that ships with Free tier is right there, working, no upgrade needed. Worth calling out because the framing is actively misleading.

6. Azure Functions reserves ?code=

The "OAuth works locally, returns 500 on production" classic, but with a twist. Symptoms:

- Click **Sign in with GitHub** on `wikidown.app/connect`. GitHub authorizes fine and 302s the browser to `https://wikidown.app/api/auth/github/callback?code=<real-code>&state=…`.
- Browser lands on a generic `wikidown.app can't currently handle this request — HTTP ERR0R 500`. URL stays on the callback path; no redirect to `/connect#gh_error=…` despite a try/catch in the callback that *always* returns a 302.
- Application Insights records nothing. The portal toggle for AI keeps reverting to off.
- The GitHub OAuth App's client secret shows **Never used**, 18+ hours after creation, despite repeated sign-in attempts.

Hours of bisecting added canary endpoints — `/api/version`, `/api/ping`, an outbound-HTTP probe at `/api/github-test` — and verified each pass directly. The callback function with no `code` param (`GET /api/auth/github/callback`) correctly redirected to `/connect#gh_error=missing_code`. Adding `?code=anything` flipped it to a 500.

The smoking-gun test was one URL in a regular browser tab: `https://wikidown.app/api/ping?code=foo` → 500. `Ping` is a one-line Function with `AuthorizationLevel.Anonymous`, no DI beyond logging, no outbound calls. The only thing that changed was the query string.

Azure Functions' HTTP host treats the query parameter `?code=` as a function-key candidate and rejects unmatched keys with HTTP 500 *before* dispatching the function — even on `AuthorizationLevel.Anonymous` routes. GitHub OAuth's redirect ALWAYS appends `?code=…`. So a GitHub OAuth callback URL on a managed Functions endpoint is structurally impossible.

Fix: stop letting `code=` reach the Functions host as a query param.

- Register the OAuth callback URL against a static HTML page served by ASWA, not by Functions. We chose `/oauth/github/return.html`.
- The page's JS reads `code` and `state` from `window.location.search`, then `fetch`-POSTs them as a JSON body to a new `GitHubExchange` Function at `/api/auth/github/exchange`. The POST URL contains no `code=`, so the host leaves it alone.
- `GitHubExchange` does the `https://github.com/login/oauth/access_token` exchange and returns `{"token":"…"}` or `{"error":"…"}`.
- The relay page redirects to `/connect#gh_token=…&gh_state=…` (or `…#gh_error=…`), preserving the existing fragment convention so `Connect.razor`'s return-handling code is unchanged.
- The PWA service worker now also early-returns on `/oauth/*` (in addition to `/api/*`) so the relay page always hits the network.

Lessons:

- **`AuthorizationLevel.Anonymous` does not actually disable the `?code=` key check on ASWA managed Functions** — at least not on the version we deployed against. The host inspects the query before checking the function's auth level.
- When App Insights toggles itself off and your function never appears in the logs, the host is rejecting the request *pre-dispatch*. The function never ran, so there's nothing to log.
- "Never used" on the OAuth App's client secret, after a day of real attempts, is a stronger signal than any 500 page. If GitHub never recorded a token-exchange POST hitting it, your callback isn't reaching the network — go upstream from the function code.
- One-URL bisection beats theory: `?param=foo` on a known-working endpoint is the cheapest possible way to falsify "the host doesn't touch query params."

Working state as of the handoff

- `https://wikidown.org/` — marketing site, live, HTTPS.
- `https://wikidown.app/` — editor PWA, live on the custom domain, OAuth-aware Connect page shipping. The ASWA default hostname `https://victorious-wave-03164381e.7.azurestaticapps.net/` is still bound as a safety net.

- All 7 Functions registered on the managed backend: `Ping`, `Version`, `GitHubProbe`, `GitHubConfig`, `AdoConfig`, `GitHubExchange`, `AdoCallback`.
- `ci.yml` builds + tests + packs on every PR; `pages.yml` deploys the marketing site; `azure-static-web-apps-*.yml` pre-builds the editor on the runner and hands it + the API source to Oryx.
- Browser test plan in `/docs/Testing/Browser-Test-Plan.md` covers the two surfaces, the API smoke tests, and the OAuth flow.

Known loose ends going into the inner loop

- Stale service worker needs evicting on the dev's machine before `/api/ping` is reachable from their regular browser.
- `GITHUB_CLIENT_ID` / `GITHUB_CLIENT_SECRET` need to be confirmed set on the SWA's Configuration → Application settings.
- The `Wikidown` GitHub OAuth App is registered with a single `Authorization callback URL`, `https://wikidown.app/oauth/github/return.html` — OAuth sign-in on the ASWA default hostname will fail with a `redirect_uri` mismatch by design. Converting to a GitHub App would allow multiple callback URLs; not worth it just for the fallback.
- ADO OAuth is stubbed — `/api/config/ado` returns empty `clientId` on purpose; ADO tab still uses PAT.
- Device flow (for CLI / MCP) is deferred until there's a real reason to ship it.

The vibe, honestly

This phase was a lot of "the docs say X; the platform does Y; the build runs but the thing 404s anyway." The pattern that paid off over and over: when the hosted builder can't do it, move the work onto the GitHub runner. When the service worker is in the way, read the service worker. When the Azure Portal banner contradicts the row below it, read the row below it.

What's in place is the boring-but-load-bearing foundation — infra, auth, deploy, two live origins, a dogfed wiki, a CLI, an MCP server, a PWA. Next loop gets to actually build features on top.